# Crime Count Forecasting per District

IT462


| Student Name | IDs | Email |
|---|---|---|
| Raghad Almutairi | 443200793 | 443200793@student.ksu.edu.sa |
| Sarah Alruwayte | 444200758 | 444200758@student.ksu.edu.sa |
| Norah Alfaheed | 444200779 | 444200779@student.ksu.edu.sa |
| Atheer Budie | 444200894 | 444200894@student.ksu.edu.sa |

Section: 2766
Supervised by: Dr. AFSHAN JAFRI

# List of Contents

# List of Table



# List of Figures

# 1. Introduction

This phase demonstrates the use of Apache Spark RDDs (Resilient Distributed Datasets)[1] to perform large-scale exploratory analysis on the Chicago Crimes dataset (952,563 clean records, 2021–2024). RDD operations were selected specifically to uncover spatial, temporal, and behavioral crime patterns that directly support the regression-based crime forecasting objective established in Phase 1.

All RDD operations are performed on the cleaned record-level dataset (952,563 records) before the weekly aggregation created in Phase 2(cleaning and preprossing phase). This ensures that exploratory analyses such as crime-type distribution, arrest rates, and district crime diversity can be computed at the incident level, which would not be possible on the aggregated dataset used later for machine learning.

# 2. Operations Overview

The table below summarizes all 7 transformations and 6 actions implemented in this phase. Each is non-trivial and directly connected to the forecasting problem.

*Table 1: Summary of all RDD transformations and actions implemented*

| # | Operation | Type | Purpose |
|---|---|---|---|
| T1 | map | Transformation | Extract (district, 1) pairs for per-district counting |
| T2 | filter | Transformation | Isolate THEFT records — the dominant crime type (22.1%) |
| T3 | reduceByKey | Transformation | Aggregate total crimes per district |
| T4 | flatMap | Transformation | Build full crime-type frequency distribution (~30 types) |
| T5 | groupByKey | Transformation | Count distinct crime types per district (behavioral mix) |
| T6 | sortByKey | Transformation | Rank districts by ID for ordered spatial reporting |
| T7 | join | Transformation | Enrich crime counts with per-district arrest data |
| A1 | count | Action | Verify 952,563 clean records loaded correctly |
| A2 | take | Action | Retrieve top 5 highest-crime districts |
| A3 | reduce | Action | Compute city-wide crime grand total (integrity check) |
| A4 | first | Action | Identify single highest-crime district (District 8) |

| # | Operation | Type | Purpose |
|---|---|---|---|
| A5 | foreach + collect | Action | Print complete ranked district listing to console |
| A6 | saveAsTextFile | Action | Persist enriched district summary report to disk |

# 3. RDD Transformations [1]

## T1: map – Extracting (District, 1) Pairs

*Table 2: Operation metadata for T1 (map) – extracting (district, 1) pairs*

| | |
|---|---|
| **Operation** | map |
| **Input RDD** | rawRDD (Row objects from clean DataFrame) |
| **Output RDD** | RDD[(Int, Int)] — (district_id, 1) |
| **Reveals** | Prepares district-keyed pairs for distributed aggregation |

The map transformation extracts a (district, 1) key-value pair from every crime record. This is the foundational step enabling all district-level aggregations, providing an exploratory basis for understanding the spatial distribution of crime volume across districts which is the primary pattern the regression model is designed to forecast.

**Code:**

```
val districtOnePairs: RDD[(Int, Int)] = rawRDD.map { row =>
  val district = row.getAs[Int]("district")
  (district, 1)
}
```

```
scala> // ================================================================
     | // ======================== TRANSFORMATIONS =======================
     | // ================================================================
     |
     | // ----------------------------------------------------------------
     | // TRANSFORMATION 1: map
     | // Purpose: Extract (district, 1) key-value pairs from each record
     | //          to prepare for per-district crime counting.
     | // ----------------------------------------------------------------
     | println("\n--- T1: map — Extract (district, 1) pairs ---")
     |
     | val districtOnePairs: RDD[(Int, Int)] = rawRDD.map { row =>
     |   val district = row.getAs[Int]("district")
     |   (district, 1)
     | }
     |
     | // Preview the transformation (action: take)
     | println("Sample (district, 1) pairs:")
     | districtOnePairs.take(5).foreach(println)

--- T1: map — Extract (district, 1) pairs ---
Sample (district, 1) pairs:
(5,1)
(6,1)
(11,1)
(10,1)
(7,1)
val districtOnePairs: org.apache.spark.rdd.RDD[(Int, Int)] = MapPartitionsRDD[37] at map at <console>:12
```

*Figure 1: Spark shell output showing T1 (map) – sample (district, 1) key-value pairs*

**Sample Output:**

```
(5,1)
(6,1)
(11,1)
(10,1)
(7,1)
```

Each record is mapped to a (district_id, 1) pair. The five sample pairs shown above confirm that records from multiple districts (5, 6, 7, 10, 11) are present in the first (initial) partition, as expected for a city-wide dataset.

## T2: filter – Isolating THEFT Records

*Table 3: Operation metadata for T2 (filter) – isolating THEFT records*

| | |
|---|---|
| **Operation** | filter |
| **Input RDD** | rawRDD (all 952,563 clean records) |
| **Output RDD** | RDD[Row] — THEFT records only |
| **Reveals** | Volume and share of the dominant crime category |

As established in Phase 2, THEFT is the most frequent crime category. This filter isolates these records for a focused sub-analysis. Analyzing the distribution of this high-volume category confirms its dominance and justifies examining THEFT patterns independently during the exploratory phase.

**Code:**

```
val theftRDD: RDD[Row] = rawRDD.filter { row =>
  row.getAs[String]("primary_type") == "THEFT"
}
val theftCount = theftRDD.count()
```

```
scala> // ------------------------------------------------------------------
     | // TRANSFORMATION 2: filter
     | // Purpose: Isolate only THEFT records — the most frequent crime
     | //          type (21.994% of dataset). Used to examine the dominant
     | //          crime pattern separately before building the full model.
     | // ------------------------------------------------------------------
     | println("\n--- T2: filter — Retain only THEFT records ---")
     |
     | val theftRDD: RDD[org.apache.spark.sql.Row] = rawRDD.filter { row =>
     |   row.getAs[String]("primary_type") == "THEFT"
     | }
     |
     | val theftCount = theftRDD.count()   // action used inside transformation block
     | println(s"Total THEFT records: $theftCount")

--- T2: filter — Retain only THEFT records ---
Total THEFT records: 210344
val theftRDD: org.apache.spark.rdd.RDD[org.apache.spark.sql.Row] = MapPartitionsRDD[38] at filter at <console>:9
val theftCount: Long = 210344
```

*Figure 2: Spark shell output showing T2 (filter) – total THEFT record count after filtering*

**Sample Output:**

```
Total THEFT records: 210,344
```

210,344 of 952,563 clean records (22.1%) are THEFT incidents, confirming it as the single dominant crime type in Chicago's 2021–2024 dataset. This is consistent with the missing-value analysis in Phase 2, which showed THEFT having a low missing rate (1.596%), confirming it is well-represented and suitable as a focal point for exploratory analysis.

## T3: reduceByKey – Total Crimes per District

*Table 4: Operation metadata for T3 (reduceByKey) – total crimes per district*

| | |
|---|---|
| **Operation** | reduceByKey |
| **Input RDD** | districtOnePairs: RDD[(Int, Int)] |
| **Output RDD** | RDD[(Int, Int)] — (district_id, total_count) |
| **Reveals** | Baseline crime volume per district — the regression target baseline |

reduceByKey efficiently aggregates the (district, 1) pairs by summing all counts for each district key across the distributed cluster. The resulting dataset provides the core spatial baseline, which reveal districts experience the highest crime burden and should receive the most forecasting attention.

**Code:**

```
val districtCrimeCount: RDD[(Int, Int)] =
  districtOnePairs.reduceByKey(_ + _)
```

```
scala> // ------------------------------------------------------------------
     | // TRANSFORMATION 3: reduceByKey
     | // Purpose: Compute total crime counts per district by summing
     | //          the (district, 1) pairs. Directly supports the regression
     | //          target: understanding which districts are highest-volume.
     | // ------------------------------------------------------------------
     | println("\n--- T3: reduceByKey — Total crimes per district ---")
     |
     | val districtCrimeCount: RDD[(Int, Int)] = districtOnePairs.reduceByKey(_ + _)
     |
     | println("Crime count per district (sample):")
     | districtCrimeCount.take(5).foreach { case (district, count) =>
     |   println(s"  District $district -> $count crimes")
     | }

--- T3: reduceByKey — Total crimes per district ---
Crime count per district (sample):
  District 20 -> 20035 crimes
  District 10 -> 40190 crimes
  District 11 -> 53691 crimes
  District 1 -> 50305 crimes
  District 31 -> 57 crimes
val districtCrimeCount: org.apache.spark.rdd.RDD[(Int, Int)] = ShuffledRDD[39] at reduceByKey at <console>:9
```

*Figure 3: Spark shell output showing T3 (reduceByKey) – sample crime after aggregation*

**Sample Output:**

District 20 -> 20,035 crimes

District 10 -> 40,190 crimes

District 11 -> 53,691 crimes

District  1 -> 50,305 crimes

District 31 ->     57 crimes

The output confirms significant spatial inequality across districts. District 8 is the highest-volume district (61,143 crimes, revealed in full by A2/A5), while District 31 records only 57 crimes, suggesting it covers a very small geographic area or has incomplete records for the study period. This heterogeneity is an important exploratory finding, confirming that crime is not evenly distributed across Chicago's districts. The forecasting model must therefore be trained on data from all districts together so it can learn each district's individual crime level from the historical records.

## T4: flatMap – Crime Type Frequency Distribution

*Table 5: Operation metadata for T4 (flatMap) – crime type frequency distribution*

| | |
|---|---|
| **Operation** | flatMap + reduceByKey |
| **Input RDD** | rawRDD (all 952,563 records) |
| **Output RDD** | RDD[(String, Int)] — (crime_type, total_count) |
| **Reveals** | Full frequency distribution across ~30 primary crime types |

flatMap emits a (crimeType, 1) pair from each record via a Seq, making explicit that each record contributes one item to the frequency count. This reveals the complete distribution of crime types across the dataset, confirming which categories dominate the dataset and providing evidence that the Phase 2 decision to preserve individual crime-type columns was well-justified.

**Code:**

```
val crimeTypeFreq: RDD[(String, Int)] = rawRDD
  .flatMap { row =>
    val crimeType = row.getAs[String]("primary_type")
    Seq((crimeType, 1))
  }
  .reduceByKey(_ + _)
```

```
scala> // ------------------------------------------------------------------
     | // TRANSFORMATION 4: flatMap
     | // Purpose: Emit one (crimeType, 1) pair per record to build a
     | //          full frequency distribution of all ~30 primary crime
     | //          types. This reveals which crime categories dominate
     | //          and should carry the most predictive weight.
     | // ------------------------------------------------------------------
     | println("\n--- T4: flatMap – Crime type frequency distribution ---")
     |
     | val crimeTypeFreq: RDD[(String, Int)] = rawRDD
     |   .flatMap { row =>
     |     val crimeType = row.getAs[String]("primary_type")
     |     Seq((crimeType, 1))   // one pair per record; flatMap opens the Seq
     |   }
     |   .reduceByKey(_ + _)
     |
     | println("Crime type frequencies (sample):")
     | crimeTypeFreq.take(8).foreach { case (ctype, cnt) =>
     |   println(f"  $ctype%-35s -> $cnt%,d")
     | }

--- T4: flatMap – Crime type frequency distribution ---
Crime type frequencies (sample):
  WEAPONS VIOLATION                   -> 34,067
  PROSTITUTION                        -> 887
  NARCOTICS                           -> 19,303
  BURGLARY                            -> 29,989
  OBSCENITY                           -> 185
  HOMICIDE                            -> 2,782
  PUBLIC PEACE VIOLATION              -> 3,135
  ROBBERY                             -> 36,963
val crimeTypeFreq: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[41] at reduceByKey at <console>:15
```

*Figure 4: Spark shell output showing T4 (flatMap) – crime type frequency distribution*

**Sample Output (8 of ~30 types):**

```
WEAPONS VIOLATION          -> 34,067
PROSTITUTION                 ->    887
NARCOTICS                  -> 19,303
BURGLARY                   -> 29,989
OBSCENITY                  ->    185
HOMICIDE                  ->  2,782
PUBLIC PEACE VIOLATION       ->  3,135
ROBBERY                   -> 36,963
```

The output reveals extreme variation in crime type frequencies — WEAPONS VIOLATION (34,067) and ROBBERY (36,963) are high-volume categories, while OBSCENITY (185) and PROSTITUTION (887) are very rare. This wide range validates the Phase 2 pivoted aggregation strategy, where each crime type becomes a separate weekly feature column. This confirms that the Phase 2 pivoted aggregation captures meaningful behavioral variation across crime categories, rather than collapsing all types into a single undifferentiated count.

## T5: groupByKey – Crime Type Mix per District

*Table 6: Operation metadata for T5 (groupByKey) – crime type mix per district*

| | |
|---|---|
| **Operation** | groupByKey |
| **Input RDD** | RDD[(Int, String)] — (district, crime_type) pairs |
| **Output RDD** | RDD[(Int, Int)] — (district, distinct_type_count) |
| **Reveals** | How many distinct crime categories each district experiences |

groupByKey collects all crime type labels for each district into an iterable. The number of distinct types per district is then computed using toSet.size. Districts with greater crime-type diversity present a more complex forecasting challenge, confirming that no district possesses a simplified crime profile; therefore, the model must account for this multi-dimensional complexity across the entire dataset. This finding supports the Phase 2 rationale for preserving all pivoted crime-type columns rather than dropping low-frequency ones.

**Code:**

```
val districtCrimeTypes: RDD[(Int, Iterable[String])] = rawRDD
  .map { row => (row.getAs[Int]("district"), row.getAs[String]("primary_type")) }
  .groupByKey()

val distinctTypesPerDistrict = districtCrimeTypes.map {
  case (district, types) => (district, types.toSet.size)
}
```

```
scala> // ------------------------------------------------------------------
     | // TRANSFORMATION 5: groupByKey
     | // Purpose: Group all crime type occurrences by district to examine
     | //          the crime-type mix per district. This supports the
     | //          behavioral feature engineering rationale from Phase 2
     | //          (pivoted crime-type columns as predictors).
     | // ------------------------------------------------------------------
     | println("\n--- T5: groupByKey — Crime types grouped by district ---")
     |
     | val districtCrimeTypes: RDD[(Int, Iterable[String])] = rawRDD
     |   .map { row =>
     |     val district  = row.getAs[Int]("district")
     |     val crimeType = row.getAs[String]("primary_type")
     |     (district, crimeType)
     |   }
     |   .groupByKey()
     |
     | // Compute distinct crime type count per district
     | val distinctTypesPerDistrict: RDD[(Int, Int)] = districtCrimeTypes.map {
     |   case (district, types) => (district, types.toSet.size)
     | }
     |
     | println("Distinct crime types per district (sample):")
     | distinctTypesPerDistrict.sortBy(_._1).take(5).foreach { case (d, n) =>
     |   println(s"  District $d -> $n distinct crime types")
     | }

--- T5: groupByKey — Crime types grouped by district ---
Distinct crime types per district (sample):
  District 1 -> 29 distinct crime types
  District 2 -> 29 distinct crime types
  District 3 -> 28 distinct crime types
  District 4 -> 29 distinct crime types
  District 5 -> 30 distinct crime types
val districtCrimeTypes: org.apache.spark.rdd.RDD[(Int, Iterable[String])] = ShuffledRDD[43] at groupByKey at <console>:11
val distinctTypesPerDistrict: org.apache.spark.rdd.RDD[(Int, Int)] = MapPartitionsRDD[44] at map at <console>:19
```

*Figure 5: Spark shell output showing T5 (groupByKey) – distinct crime type count per district)*

**Sample Output:**

District 1 -> 29 distinct crime types

District 2 -> 29 distinct crime types

District 3 -> 28 distinct crime types

District 4 -> 29 distinct crime types

District 5 -> 30 distinct crime types

Every sampled district experiences 28–30 distinct crime types, confirming that the behavioral diversity of crime is consistent across Chicago's districts. This means no district can be treated as a special case with a simplified crime profile, supporting the Phase 2 decision to keep the full set of pivoted crime-type columns for all districts.

## T6: sortByKey – Districts in Ascending Order

*Table 7: Operation metadata for T6 (sortByKey) – districts in ascending order*

| | |
|---|---|
| **Operation** | sortByKey |
| **Input RDD** | districtCrimeCount: RDD[(Int, Int)] |
| **Output RDD** | RDD[(Int, Int)] — sorted by district ID ascending |
| **Reveals** | Ordered view of crime volumes for readable district-level reporting |

sortByKey orders the district crime count RDD by district identifier in ascending order. This produces a clean sequential listing of the districts present in the dataset, making it straightforward to read and compare crime volumes across all districts in order.

**Code:**

```
val sortedDistrictCrimes: RDD[(Int, Int)] =
  districtCrimeCount.sortByKey(ascending = true)
```

```
scala>
     | // ------------------------------------------------------------------
     | // TRANSFORMATION 6: sortByKey
     | // Purpose: Sort districts in ascending order to produce a clean
     | //          ranked view of total crime volumes, supporting the
     | //          spatial component of the forecasting problem.
     | // ------------------------------------------------------------------
     | println("\n--- T6: sortByKey — Districts ranked by ID (ascending) ---")
     |
     | val sortedDistrictCrimes: RDD[(Int, Int)] = districtCrimeCount.sortByKey(ascending = true)
     |
     | println("Sorted district crime counts:")
     | sortedDistrictCrimes.take(10).foreach { case (district, count) =>
     |   println(f"  District $district%2d -> $count%,d crimes")
     | }

--- T6: sortByKey — Districts ranked by ID (ascending) ---
Sorted district crime counts:
  District  1 -> 50,305 crimes
  District  2 -> 47,391 crimes
  District  3 -> 48,304 crimes
  District  4 -> 54,444 crimes
  District  5 -> 39,681 crimes
  District  6 -> 58,262 crimes
  District  7 -> 42,026 crimes
  District  8 -> 61,143 crimes
  District  9 -> 42,006 crimes
  District 10 -> 40,190 crimes
val sortedDistrictCrimes: org.apache.spark.rdd.RDD[(Int, Int)] = ShuffledRDD[52] at sortByKey at <console>:10
```

*Figure 6: Spark shell output showing T6 (sortByKey) – district crime counts sorted by district ID*

**Sample Output (first 8 districts):**

```
District  1 -> 50,305 crimes
District  2 -> 47,391 crimes
District  3 -> 48,304 crimes
District  4 -> 54,444 crimes
District  5 -> 39,681 crimes
District  6 -> 58,262 crimes
District  7 -> 42,026 crimes
District  8 -> 61,143 crimes
  ...
```

The sorted output shows that Districts 1–8 all carry substantial crime volumes (39,681–61,143), with no district in the low end of the range among this group. This confirms that the central Chicago districts require consistent forecasting attention. Note that the dataset contains 23 active district IDs (Districts 13, 21, 23 are not present, consistent with Chicago PD's reorganization), plus District 31 with a negligible 57 records.

## T7: join – Enriching Districts with Arrest Rate

*Table 8: Operation metadata for T7 (join) – enriching districts with arrest rate*

| | |
|---|---|
| **Operation** | join |
| **Input RDDs** | districtCrimeCount + districtArrestCount |
| **Output RDD** | RDD[(Int, (Int, Int))] — (district, (total_crimes, arrests)) |
| **Reveals** | Per-district arrest rate — a proxy for enforcement effectiveness |

The join transformation merges two independently computed RDDs on the district key: total crime count and total arrest count. The derived arrest rate (arrests ÷ total crimes × 100) reveals where enforcement outcomes differ significantly across Chicago. Districts with low arrest rates despite high crime volumes represent areas where reactive policing is least effective, making proactive forecasting most valuable.

**Code:**

```
val districtArrestCount: RDD[(Int, Int)] = rawRDD
  .filter { row => row.getAs[Boolean]("arrest") }
  .map    { row => (row.getAs[Int]("district"), 1) }
  .reduceByKey(_ + _)

val districtEnriched = districtCrimeCount.join(districtArrestCount)
```

```
scala> // ------------------------------------------------------------------
     | // TRANSFORMATION 7: join[(Int, (Int, Int))] =
     | // Purpose: Enrich district-level crime counts with the per-district
     | //          arrest counts to compute an arrest rate per district.
     | //          This derived metric helps reveal districts where crimes
     | //          are cleared quickly vs. those with low enforcement outcome.)) =>
     | // ------------------------------------------------------------------ests)) =>
     | println("\n--- T7: join — Enrich district counts with arrest counts ---") | Rate: $rate%.1f%%")
     |
     | // Arrest count per district
     | val districtArrestCount: RDD[(Int, Int)] = rawRDD
     |   .filter  { row => row.getAs[Boolean]("arrest") }
     |   .map     { row => (row.getAs[Int]("district"), 1) }
     |   .reduceByKey(_ + _)
     |
     | // Join total crime counts with arrest counts
     | val districtEnriched: RDD[(Int, (Int, Int))] =
     |   districtCrimeCount.join(districtArrestCount)
     |
     | println("Joined (district -> (total_crimes, arrests)) sample:")
     | districtEnriched.sortByKey().take(5).foreach { case (d, (total, arrests)) =>
     |   val rate = (arrests.toDouble / total * 100)
     |   println(f"  District $d%2d | Crimes: $total%,6d | Arrests: $arrests%,5d | Rate: $rate%.1f%%")
     | }

--- T7: join — Enrich district counts with arrest counts ---
Joined (district -> (total_crimes, arrests)) sample:
  District  1 | Crimes: 50,305 | Arrests: 8,036 | Rate: 16.0%
  District  2 | Crimes: 47,391 | Arrests: 4,093 | Rate: 8.6%
  District  3 | Crimes: 48,304 | Arrests: 5,127 | Rate: 10.6%
  District  4 | Crimes: 54,444 | Arrests: 5,113 | Rate: 9.4%
  District  5 | Crimes: 39,681 | Arrests: 5,437 | Rate: 13.7%
val districtArrestCount: org.apache.spark.rdd.RDD[(Int, Int)] = ShuffledRDD[55] at reduceByKey at <console>:14
val districtEnriched: org.apache.spark.rdd.RDD[(Int, (Int, Int))] = MapPartitionsRDD[58] at join at <console>:18
```

*Figure 7: Spark shell output showing T7 (join) – per-district*

**Sample Output (Districts 1–5):**

District 1 | Crimes: 50,305 | Arrests: 8,036 | Rate: 16.0%

District 2 | Crimes: 47,391 | Arrests: 4,093 | Rate: 8.6%

District 3 | Crimes: 48,304 | Arrests: 5,127 | Rate: 10.6%

District 4 | Crimes: 54,444 | Arrests: 5,113 | Rate: 9.4%

District 5 | Crimes: 39,681 | Arrests: 5,437 | Rate: 13.7%

The actual arrest rates are remarkably low, ranging from roughly 8.6% (District 2) to 16.0% (District 1) in this sample. This means the vast majority of crimes across all districts do not result in an arrest, underscoring the systemic need for proactive, forecasting-driven resource allocation rather than purely reactive policing. The wide variation in arrest rates (nearly 2× difference between Districts 1 and 2) also suggests structural differences in crime composition and policing strategy between districts, which is a nuance that the Phase 2 district_indexed encoding accounts for by treating each district as a unique category rather than a point on a numeric scale.

# 4. RDD Actions [1]

## A1: count – Total Record Verification

count is applied to rawRDD immediately after loading to confirm that exactly 952,563 clean records were loaded across all Spark partitions. This integrity check verifies that the Phase 2 cleaning pipeline (removing 19,302 records with missing geographic fields from the original 971,865) is correctly reproduced before any RDD transformations are applied.

**Code:**

```
val totalRecords = rawRDD.count()
println(s"Total clean records: $totalRecords")
```

```
scala> // ================================================================
     | // ============================ ACTIONS ===========================
     | // ================================================================
     |
     | // ----------------------------------------------------------------
     | // ACTION 1: count
     | // Purpose: Confirm total number of records after cleaning and
     | //          verify scale of data requiring Spark's distributed engine.
     | // ----------------------------------------------------------------
     | println("\n--- A1: count – Total records after cleaning ---")
     |
     | val totalRecords = rawRDD.count()
     | println(s"Total clean records: $totalRecords")

--- A1: count – Total records after cleaning ---
Total clean records: 952563
val totalRecords: Long = 952563
```

*Figure 8: Spark shell output showing A1 (count) – total clean record count*

**Output:**

```
Total clean records: 952,563
```

## A2: take – Top 5 Highest-Crime Districts

take(5) retrieves the top 5 districts ranked by total crime volume after sorting the districtCrimeCount RDD in descending order. This confirms which districts carry the greatest crime burden and should receive the highest forecasting priority, as model errors for these districts have the largest operational impact on resource allocation decisions.

**Code:**

```
val top5Districts = districtCrimeCount
  .sortBy(_._2, ascending = false)
  .take(5)
```

```
scala> // ------------------------------------------------------------
     | // ACTION 2: take
     | // Purpose: Inspect the top 5 highest-crime districts by retrieving
     | //          the sorted output. Confirms which districts should receive
     | //          the most attention in patrol resource allocation.
     | // ------------------------------------------------------------
     | println("\n--- A2: take — Top 5 highest-crime districts ---")
     |
     | val top5Districts = districtCrimeCount
     |   .sortBy(_._2, ascending = false)
     |   .take(5)
     |
     | println("Top 5 districts by crime volume:")
     | top5Districts.foreach { case (district, count) =>
     |   println(f"  District $district%2d -> $count%,d crimes")
     | }

--- A2: take — Top 5 highest-crime districts ---
Top 5 districts by crime volume:
  District  8 -> 61,143 crimes
  District  6 -> 58,262 crimes
  District 12 -> 56,268 crimes
  District  4 -> 54,444 crimes
  District 11 -> 53,691 crimes
val top5Districts: Array[(Int, Int)] = Array((8,61143), (6,58262), (12,56268), (4,54444), (11,53691))
```

*Figure 9: Spark shell output showing A2 (take) – top 5 highest-crime districts*

**Output:**

```
District  8 -> 61,143 crimes
District  6 -> 58,262 crimes
District 12 -> 56,268 crimes
District  4 -> 54,444 crimes
District 11 -> 53,691 crimes
```

District 8 leads with 61,143 crimes across 2021–2024, followed closely by Districts 6 (58,262) and 12 (56,268). The top 5 districts together account for approximately 283,808 incidents(just under 30% of all

clean records) despite representing fewer than 25% of all active districts. This heavy spatial concentration means that correctly forecasting these 5 districts is disproportionately important for the project's public safety goal.

## A3: reduce – City-Wide Grand Total

reduce sums all per-district crime counts into a single city-wide total using a distributed addition. The result is compared directly against the rawRDD.count() value from A1 as a data integrity check: if the two numbers match, it confirms that every clean record belongs to exactly one valid district and that no records were lost or double-counted during the reduceByKey aggregation (T3).

**Code:**

```
val cityTotal = districtCrimeCount
  .map(_._2)
  .reduce(_ + _)
println(s"City-wide total crimes (2021–2024): $cityTotal")
```

```
scala> // ------------------------------------------------------------------
     | // ACTION 3: reduce
     | // Purpose: Compute the grand total of all crimes across the entire
     | //          city (2021–2024) using a distributed sum. This gives a
     | //          single city-wide baseline to contextualize district volumes.
     | // ------------------------------------------------------------------
     | println("\n--- A3: reduce – City-wide total crime count ---")
     |
     | val cityTotal = districtCrimeCount
     |   .map(_._2)
     |   .reduce(_ + _)
     |
     | println(s"City-wide total crimes (2021–2024): $cityTotal")

--- A3: reduce – City-wide total crime count ---
City-wide total crimes (2021–2024): 952563
val cityTotal: Int = 952563
```

*Figure 10: Spark shell output showing A3 (reduce) – city-wide grand total*

**Output:**

```
City-wide total crimes (2021–2024): 952,563
```

The city-wide total (952,563) exactly matches the rawRDD.count() result from A1, confirming full data integrity across the distributed pipeline. Every clean record is assigned to exactly one district, and no records were lost or duplicated during the reduceByKey aggregation.

## A4: first – Highest-Crime District Identification

first retrieves the single top-ranked (district, crime_count) pair after sorting the districtCrimeCount RDD in descending order. This identifies the district that demands the most precise forecasting, as prediction errors there carry the greatest operational consequence for patrol scheduling and resource deployment.

**Code:**

```
val highestCrimeDistrict = districtCrimeCount
  .sortBy(_._2, ascending = false)
  .first()
println(s"Highest-crime district: District ${highestCrimeDistrict._1} with ${highestCrimeDistrict._2} crimes")
```

```
scala> // ------------------------------------------------------------
     | // ACTION 4: first
     | // Purpose: Retrieve the district with the highest weekly crime
     | //          volatility (std dev). High volatility districts are harder
     | //          to forecast and signal where the model needs more features.
     | // ------------------------------------------------------------
     | println("\n--- A4: first — District with highest crime count (sorted desc) ---")
     |
     | val highestCrimeDistrict = districtCrimeCount
     |   .sortBy(_._2, ascending = false)
     |   .first()
     |
     | println(s"Highest-crime district: District ${highestCrimeDistrict._1} with ${highestCrimeDistrict._2} crimes")

--- A4: first — District with highest crime count (sorted desc) ---
Highest-crime district: District 8 with 61143 crimes
val highestCrimeDistrict: (Int, Int) = (8,61143)
```

*Figure 11: Spark shell output showing A4 (first) – highest-crime district*

**Output:**

```
Highest-crime district: District 8 with 61,143 crimes
```

District 8 is the single highest-crime district in Chicago for the 2021–2024 period with 61,143 incidents. Located on the Southwest Side of Chicago, District 8 covers a large geographic area including multiple residential neighborhoods. Accurate weekly forecasts for District 8 are therefore the most operationally critical output of the model.

## A5: foreach + collect – Full District Ranking

collect() pulls all 23 active district entries from the distributed RDD to the driver node, after which foreach iterates over them and prints each district's total crime count in descending order. The use of collect() is safe here because the RDD contains only 23 rows and therefore far below any memory threshold.

**Code:**

```
districtCrimeCount
  .sortBy(_._2, ascending = false)
  .collect()
  .foreach { case (district, count) =>
    println(f"District $district%2d -> $count%,d crimes")
  }
```

```
scala> // ------------------------------------------------------------------
     | // ACTION 5: foreach
     | // Purpose: Print a full ranked summary of all districts by crime
     | //          volume. Provides the spatial baseline needed to interpret
     | //          regression predictions in Phase 5.
     | // ------------------------------------------------------------------
     | println("\n--- A5: foreach — Full district ranking by crime volume ---")
     |
     | println("All districts ranked by crime volume (descending):")
     | districtCrimeCount
     |   .sortBy(_._2, ascending = false)
     |   .collect()                          // small RDD: only ~31 districts
     |   .foreach { case (district, count) =>
     |     println(f"  District $district%2d -> $count%,d crimes")
     |   }

--- A5: foreach — Full district ranking by crime volume ---
All districts ranked by crime volume (descending):
  District  8 -> 61,143 crimes
  District  6 -> 58,262 crimes
  District 12 -> 56,268 crimes
  District  4 -> 54,444 crimes
  District 11 -> 53,691 crimes
  District  1 -> 50,305 crimes
  District 25 -> 49,408 crimes
  District 19 -> 48,423 crimes
  District  3 -> 48,304 crimes
  District 18 -> 48,209 crimes
  District  2 -> 47,391 crimes
  District  7 -> 42,026 crimes
  District  9 -> 42,006 crimes
  District 10 -> 40,190 crimes
  District  5 -> 39,681 crimes
  District 16 -> 34,805 crimes
  District 14 -> 33,555 crimes
  District 15 -> 32,922 crimes
  District 24 -> 32,921 crimes
  District 22 -> 30,435 crimes
  District 17 -> 28,082 crimes
  District 20 -> 20,035 crimes
  District 31 -> 57 crimes
```

*Figure 12: Spark shell output showing A5 (foreach + collect) – complete ranked list*

**Full Output:**

```
District  8 -> 61,143 crimes
District  6 -> 58,262 crimes
District 12 -> 56,268 crimes
District  4 -> 54,444 crimes
District 11 -> 53,691 crimes
District  1 -> 50,305 crimes
District 25 -> 49,408 crimes
District 19 -> 48,423 crimes
District  3 -> 48,304 crimes
District 18 -> 48,209 crimes
District  2 -> 47,391 crimes
District  7 -> 42,026 crimes
District  9 -> 42,006 crimes
District 10 -> 40,190 crimes
District  5 -> 39,681 crimes
District 16 -> 34,805 crimes
District 14 -> 33,555 crimes
District 15 -> 32,922 crimes
District 24 -> 32,921 crimes
District 22 -> 30,435 crimes
District 17 -> 28,082 crimes
District 20 -> 20,035 crimes
District 31 ->      57 crimes
```

The complete ranking reveals that crime is heavily concentrated in a small number of districts — Districts 8, 6, and 12 alone account for a disproportionate share of all incidents, while District 31's 57 records suggest it represents a decommissioned or administrative boundary rather than an operational policing area. This spatial overview provides important context for interpreting the forecasting model's per-district results in later phases.

## A6: saveAsTextFile – Persisting District Enrichment Report

saveAsTextFile writes the enriched district-level summary (total crimes, arrest counts, and computed arrest rate) to disk as a formatted text file, making it available for offline inspection without a running Spark session. The saved report documents the complete per-district profile covering total crime volume, total arrests, and the derived arrest rate across all 23 districts

**Code:**

```
districtEnriched
  .sortByKey()
  .map { case (district, (total, arrests)) =>
    val rate = arrests.toDouble / total * 100
    f"District $district%2d | Crimes: $total%,6d | Arrests: $arrests%,5d | Rate: $rate%.2f%%"
  }
  .saveAsTextFile("/Users/raghadfares/Desktop/BigData_Phase3/district_crime_summary")
```

```
scala> // ------------------------------------------------------------
     | // ACTION 6: saveAsTextFile
     | // Purpose: Persist the enriched district-level summary (crime counts
     | //          + arrest rates) to disk as a readable text report for
     | //          inclusion in the final project submission.
     | // ------------------------------------------------------------
     | println("\n--- A6: saveAsTextFile — Save district enrichment summary ---")
     |
     | val outputPath = "/Users/raghadfares/Desktop/BigData_Phase3/district_crime_summary"
     |
     | districtEnriched
     |   .sortByKey()
     |   .map { case (district, (total, arrests)) =>
     |     val rate = arrests.toDouble / total * 100
     |     f"District $district%2d | Total Crimes: $total%,6d | Arrests: $arrests%,5d | Arrest Rate: $rate%.2f%%"
     |   }
     |   .saveAsTextFile(outputPath)
     |
     | println(s"District summary saved to: $outputPath")

--- A6: saveAsTextFile — Save district enrichment summary ---
District summary saved to: /Users/raghadfares/Desktop/BigData_Phase3/district_crime_summary
val outputPath: String = /Users/raghadfares/Desktop/BigData_Phase3/district_crime_summary
```

*Figure 13: Spark shell output showing A6 (saveAsTextFile)*

**Output:**

```
District summary saved to: /Users/raghadfares/Desktop/BigData_Phase3/district_crime_summary
```

## Year-over-Year Crime Trend per District

As an additional analysis, (district, year) composite keys were created using map, then reduceByKey was applied to aggregate yearly crime counts per district. Filtering on District 11 then reveals the year-over-year trajectory which is a direct validation of temporal trends used to motivate the lag feature engineered in Phase 2.

**Code:**

```
val yearlyDistrictTrend: RDD[((Int, Int), Int)] = rawRDD
  .map { row =>
    val district = row.getAs[Int]("district")
    val year     = row.getAs[Int]("year")
    ((district, year), 1)
  }
  .reduceByKey(_ + _)

yearlyDistrictTrend
  .filter { case ((district, _), _) => district == 11 }
  .sortBy { case ((_, year), _) => year }
  .collect()
  .foreach { case ((district, year), count) =>
    println(f"District $district | Year $year | Crimes: $count%,d")
  }
```

```
     | // Purpose: Emit (district, year, 1) triples and aggregate to reveal
     | //          whether crime is rising or falling per district — a key
     | //          insight for validating temporal forecasting patterns.
     | // ============================================================
     | println("\n--- BONUS: Year-over-Year crime trend per district ---")
     |
     | val yearlyDistrictTrend: RDD[((Int, Int), Int)] = rawRDD
     |   .map { row =>
     |     val district = row.getAs[Int]("district")
     |     val year     = row.getAs[Int]("year")
     |     ((district, year), 1)
     |   }
     |   .reduceByKey(_ + _)
     |
     | println("Yearly crime trend per district (District 11 example):")
     | yearlyDistrictTrend
     |   .filter { case ((district, _), _) => district == 11 }
     |   .sortBy { case ((_, year), _) => year }
     |   .collect()
     |   .foreach { case ((district, year), count) =>
     |     println(f"  District $district | Year $year | Crimes: $count%,d")
     |   }

--- BONUS: Year-over-Year crime trend per district ---
Yearly crime trend per district (District 11 example):
  District 11 | Year 2021 | Crimes: 12,888
  District 11 | Year 2022 | Crimes: 12,989
  District 11 | Year 2023 | Crimes: 14,253
  District 11 | Year 2024 | Crimes: 13,561
val yearlyDistrictTrend: org.apache.spark.rdd.RDD[((Int, Int), Int)] = ShuffledRDD[84] at reduceByKey at <console>:16
```

*Figure 14: Spark shell output showing Bonus (map + reduceByKey) – crime trend for District 11*

**Output (District 11):**

> District 11 | Year 2021 | Crimes: 12,888
>
> District 11 | Year 2022 | Crimes: 12,989
>
> District 11 | Year 2023 | Crimes: 14,253
>
> District 11 | Year 2024 | Crimes: 13,561

The District 11 trend is notably non-linear: crime rose from 12,888 (2021) to a peak of 14,253 (2023) before declining to 13,561 in 2024. This non-monotonic pattern highlights why simple linear trend models are insufficient and supports the use of lagged features (prev_week_total) and seasonal indicators (month, week_no) engineered in Phase 2. these features allow the model to adapt to changing momentum rather than assuming a constant trend direction.

# 5. Analysis Summary and Key Insights

## 5.1 Spatial Distribution of Crime

The reduceByKey (T3), sortByKey (T6), and foreach/collect (A5) operations collectively reveal a highly unequal distribution of crime across Chicago's 23 active districts. District 8 leads with 61,143 crimes, while the bottom third of districts record fewer than 35,000 incidents each. The top 5 districts account for roughly 30% of all city-wide incidents despite representing just 22% of active districts. This strong spatial heterogeneity confirms that crime is not evenly distributed across Chicago's districts, and that the forecasting model must be trained on all districts together so it can learn each district's individual crime level from the historical records.

## 5.2 Crime Type Dominance and Behavioral Features

The flatMap frequency distribution (T4) confirms that the dataset spans approximately 30 distinct crime types with extremely varied frequencies — from THEFT (210,344 records, 22.1%) and BATTERY down to rare types such as OBSCENITY (185) and PROSTITUTION (887). This wide variation validates the Phase 2 decision to use a pivoted aggregation rather than just a total count, since collapsing all crime types into one number would hide meaningful differences in the weekly crime mix across districts.

## 5.3 Consistent Behavioral Complexity Across Districts

The groupByKey analysis (T5) shows that districts 1–5 each experience 28–30 distinct crime types. This near-uniform diversity means the regression model's full set of behavioral feature columns (one per crime type from the Phase 2 pivot) is informative for every district, supporting a single unified model architecture rather than separate district-specific sub-models. A single model is also preferred for data efficiency, given that the aggregated dataset contains only 4,667 weekly rows.

## 5.4 Critically Low Arrest Rates

The join operation (T7) reveals that arrest rates across sampled districts are very low — ranging from approximately 8.6% (District 2) to 16.0% (District 1). This means that for every 10 crimes recorded, only 1–2 result in an arrest. This finding powerfully strengthens the project's core motivation: because reactive policing (waiting for crimes to occur and then making arrests) is structurally limited, a proactive forecasting system that predicts crime volumes in advance offers meaningful value for patrol planning and resource pre-deployment.

## 5.5 Non-Linear Temporal Trends Require Lag Features

The year-over-year analysis demonstrates that crime trends are non-linear even within a single district. District 11's trajectory (12,888 → 12,989 → 14,253 → 13,561) shows a rise-then-fall pattern rather than a monotonic trend. This validates the Phase 2 lag feature (prev_week_total), which captures recent momentum rather than long-run direction. It also motivates the inclusion of month and week_no seasonal features to help the model detect cyclical patterns within years.

## 5.6 Summary: RDD Insights Supporting the Model

Collectively, the RDD operations confirm five structural properties of the dataset that directly inform the machine learning pipeline planned for later Phases:

- Strong spatial heterogeneity across districts → the model must be trained on all districts together to learn each district's individual crime level.
- 22.1% of incidents are THEFT, with extreme frequency variation across ~30 crime types → behavioral pivot features add predictive value beyond simple totals
- 28–30 distinct crime types appear in every district → a single unified model is appropriate
- Arrest rates of 8.6%–16% confirm limited reactive policing effectiveness → proactive forecasting is well-motivated
- Non-linear year-over-year trends → lag and seasonal features are necessary for accurate short-term prediction

## 6. Reference

[1] TutorialsPoint, "Apache Spark Tutorial," TutorialsPoint (I) Pvt. Ltd., 2015. [Online]. Available: https://www.tutorialspoint.com/apache_spark/apache_spark_tutorial.pdf. [Accessed: 13-Mar-2026].